

TECHNICAL DOCUMENT 7: VISUAL STUDIO CODE & WINDSURF IDE INTEGRATION GUIDE

Running SonarQube, PMD, ESLint, and Codeium/Windsurf Locally in your IDE

This guide details how to integrate **ESLint**, **SonarQube (via SonarLint)**, **PMD**, and the **Codeium AI Assistant / Windsurf IDE** directly inside your local environment.

While the **AI Code Quality Analyzer Web App** provides a unified bento-style dashboard for aggregated reporting and security auditing, setting up these tools locally in your editor allows you to detect violations, refactor styles, and run AI suggestions **on-the-fly as you type**.

1. Codeium vs. Windsurf IDE: What's the Difference?

If you are wondering whether to use **Codeium** or **Windsurf**, here is the relationship between the two:

- **Codeium** is the parent AI technology company. They offer a highly popular AI coding extension that integrates into existing editors like **VS Code**, **JetBrains**, and **Vim**.
 - **Windsurf** is the next-generation, AI-first IDE built **by Codeium**. It is a standalone editor (based on the VS Code open-source core) that has Codeium's powerful **Cascade AI agent** deeply baked into its core architecture rather than acting as a standard sidebar extension.
 - **Which should you use?**
 - If you want to keep using your existing, fully-configured **VS Code** workspace, install the **Codeium Extension** (see Section 5).
 - If you want a native, deeply-integrated AI-agent experience where the AI can read, edit, run terminal commands, and debug your code automatically (similar to Cline or Devin), download the **Windsurf IDE**.
-

2. Using the Windsurf IDE (Powered by Codeium)

Because Windsurf is built on the VS Code core, it supports all your favorite VS Code themes, keybindings, and extensions (including ESLint and SonarLint).

Step 1: Download & Install

1. Go to the official website: codeium.com/windsurf.
2. Download the installer for your operating system (macOS, Windows, or Linux).
3. Install and launch the application.

Step 2: Import your Settings

Upon launching Windsurf for the first time, it will offer to import all your VS Code extensions, configuration settings, and keybindings with a single click.

Step 3: Using "Cascade" (The Agent Mode)

1. Open your project folder in Windsurf.
 2. Open the **Cascade** panel on the right side or press **Ctrl+L** (or **Cmd+L**).
 3. You can use two modes:
 - **Chat Mode**: Ask questions, explain complexity, or suggest linter rules.
 - **Write/Agent Mode**: Give Cascade permission to automatically edit files, resolve SonarQube code smells, fix ESLint warnings on-the-fly, or write testing code.
-

3. ESLint Setup in VS Code / Windsurf

ESLint checks your JavaScript/TypeScript code for syntax, style violations, and scoping issues.

Step 1: Install the Extension

1. Open the Extensions View (**Ctrl+Shift+X** or **Cmd+Shift+X**).
2. Search for **ESLint** (by Dirk Baeumer).
3. Click **Install**.

Step 2: Configure Auto-Fix on Save

To make your editor automatically resolve style issues whenever you save a file, create or edit your workspace settings file:

- File path: `.vscode/settings.json`
- Add the following config:

```
{
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "always"
  },
  "eslint.validate": [
    "javascript",
    "javascriptreact",
    "typescript",
    "typescriptreact"
  ]
}
```

Step 3: View ESLint Reports

- **Inline Highlights**: Violations will be marked with red (errors) or yellow (warnings) squiggly underlines. Hover over the underlined code to see the rule ID (e.g., `eqeqeq`, `@typescript-eslint/no-unused-vars`).
 - **The Problems Tab**: Press **Ctrl+Shift+M** (or **Cmd+Shift+M**) to open the **Problems Panel** at the bottom of the editor. This displays a comprehensive list of all ESLint violations across the open files.
-

4. SonarQube Setup in VS Code / Windsurf (SonarQube Synchronizer & QPack)

If you are using a locked-down development container or a custom VS Code extension marketplace that doesn't host SonarLint directly, you can achieve full SonarQube auditing and code metrics using the **SonarQube Synchronizer** and the **Quality/Metric Extension Pack (QPack)**.

Step 1: Install the Extensions

1. Open the Extensions View (**Ctrl+Shift+X** or **Cmd+Shift+X**).
2. Search for and install:
 - **SonarQube Synchronizer**: Connects your local project workspace to your team's SonarQube server and synchronizes issue highlights directly onto your local source files.
 - **Quality/Metric Extension Pack (QPack)**: A comprehensive pack featuring rulesets and structural estimators for measuring code density, nested complex expressions, cognitive complexity, and architectural compliance.

Step 2: Configure SonarQube Synchronizer

To sync your reports locally, configure the connection to your SonarQube instance inside your workspace:

- File path: `.vscode/settings.json`
- Add the configuration:

```
{
  "sonarqube.serverUrl": "https://your-sonarqube-instance.com",
  "sonarqube.token": "your-sonarqube-user-auth-token",
  "sonarqube.projectKey": "ai-code-quality-analyzer"
}
```

Step 3: View QPack and SonarQube Reports in your Editor

- **Active Sync Marks**: The **SonarQube Synchronizer** pulls server-side issues and overlays them directly onto your active lines of code with detailed hover descriptions.
- **Complexity Badges**: **QPack** adds real-time metrics above your functions showing Cyclomatic and Cognitive complexity numbers directly.
- **Unified Problems View**: All issues detected by the linter/synchronizer flow immediately into the VS Code **Problems panel** (**Ctrl+Shift+M** or **Cmd+Shift+M**).

5. Codeium AI Extension Setup in Standard VS Code

If you prefer to stay in standard VS Code instead of switching to Windsurf, you can install Codeium as an extension.

Step 1: Install the Extension

1. Open the Extensions View (**Ctrl+Shift+X** or **Cmd+Shift+X**).
2. Search for **Codeium** (by Codeium).
3. Click **Install**.

Step 2: Authenticate

1. After installation, a prompt will appear asking you to log in.
2. Click **Sign In with Codeium** (this will open your browser).
3. Register for a free account or log in. Once authenticated, copy the token or let the browser redirect back to VS Code.

Step 3: How to Use Codeium

- **AI Auto-Completions:** As you write code, Codeium will suggest grayed-out ghost-text completions. Press **Tab** to accept them.
- **Codeium Chat:** Click the **Codeium Logo** in the side panel or press **Ctrl+Alt+C** (or **Cmd+Alt+C**) to open the interactive chatbot. Ask it to:
 - "Refactor this function to reduce cyclomatic complexity."
 - "Is there a security flaw in this SQL statement?"
- **Inline Refactoring:** Highlight any code snippet, press **Ctrl+I** (or **Cmd+I**), and type a custom command like **/refactor** or **/explain** to modify code on the fly.

6. PMD Setup in VS Code / Windsurf

PMD focuses on code complexity, style violations, and design error-proneness.

Step 1: Install the Extension

1. Search for the **Apex PMD** or **PMD VSCode** extension in the marketplace.
2. If you are analyzing other languages, you can also run PMD as a build script or use extensions like **CodeMetrics** to view cyclomatic/cognitive complexity scores directly above your functions.

Step 2: Configuration

Configure the path to your rulesets inside **.vscode/settings.json** to enable rules like naming standards, nested loops limits, or complexity constraints.

7. Overview of Reports in your IDE

Here is how to map what you see in the **AI Code Quality Analyzer Dashboard** to the panels in **VS Code / Windsurf**:

Quality Analyzer Metric	Where to find it in VS Code / Windsurf	Panel / Interface Shortcut
ESLint Warnings	Problems Panel & Inline Yellow Lines	Ctrl+Shift+M or Cmd+Shift+M
SonarQube Bugs & Smells	SonarQube Synchronizer & Problems Panel	Ctrl+Shift+M or Cmd+Shift+M
Complexity Ratings	Quality/Metric Extension Pack (QPack) or CodeMetrics	Displayed as code-lens above methods

Quality Analyzer Metric	Where to find it in VS Code / Windsurf	Panel / Interface Shortcut
AI Refactoring Proposals	Codeium Cascade or Codeium Chat	Ctrl+L (Windsurf Cascade) or Ctrl+Alt+C (VS Code Chat)
Build & Compilation Errors	VS Code Terminal / Output logs	`Ctrl+`` (Terminal)